

Titanic Data Set

Passenger data from people onboard the Titanic:

```
df_titanic.head()
```

	PassengerId	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	892	3	Kelly, Mr. James	male	34.5	0	0	330911	7.8292	NaN	Q
1	893	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272	7.0000	NaN	S
2	894	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276	9.6875	NaN	Q
3	895	3	Wirz, Mr. Albert	male	27.0	0	0	315154	8.6625	NaN	S
4	896	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3101298	12.2875	NaN	S

```
df_titanic.dtypes
```

```
PassengerId      int64
Pclass            int64
Name              object
Sex               object
Age              float64
SibSp             int64
Parch             int64
Ticket            object
Fare              float64
Cabin             object
Embarked          object
dtype: object
```

Iterrows

```
df = df_titanic[["Name", "Sex", "Age"]].copy()  
df
```

	Name	Sex	Age
0	Kelly, Mr. James	male	34.5
1	Wilkes, Mrs. James (Ellen Needs)	female	47.0
2	Myles, Mr. Thomas Francis	male	62.0
3	Wirz, Mr. Albert	male	27.0
4	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0

You can use the `iterrows()` method to go through the rows of a dataframe using a for loop.

Iterrows

Returns two loop variables

Index of df

Row as series



```
for index, row in df.iterrows():  
  
    print("-----")  
    print(index)  
    print(row)  
    print("-----")
```

Iterrows

```
for index, row in df.iterrows():  
  
    print("-----")  
    print(index)  
    print(row)  
    print("-----")
```

0

Name Kelly, Mr. James

Sex male

Age 34.5

Name: 0, dtype: object

1

Name Wilkes, Mrs. James (Ellen Needs)

Sex female

Age 47

Name: 1, dtype: object

~

Iterrows

```
df = df_titanic[["Name", "Sex", "Age"]].copy()  
df
```

	Name	Sex	Age
0	Kelly, Mr. James	male	34.5
1	Wilkes, Mrs. James (Ellen Needs)	female	47.0
2	Myles, Mr. Thomas Francis	male	62.0
3	Wirz, Mr. Albert	male	27.0
4	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0

Let's say I want to create a dictionary where I have an entry for each passenger where the key is the last name and the value is the age.

Iterrows

	Name	Sex	Age
0	Kelly, Mr. James	male	34.5
1	Wilkes, Mrs. James (Ellen Needs)	female	47.0
2	Myles, Mr. Thomas Francis	male	62.0
3	Wirz, Mr. Albert	male	27.0
4	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0

```
D = {}
```

```
for index, row in df.iterrows():
```

```
    name = row["Name"]
```

```
    age = row["Age"]
```

```
    last_name = name.replace(", ", "").split(" ")[0]
```

```
    D[last_name] = age
```

```
D
```

```
{'Hirvonen': 22.0, 'Kelly': 34.5, 'Myles': 62.0, 'Wilkes': 47.0, 'Wirz': 27.0}
```

Pivot Table

Pivot Table

A pivot table is a **table of values** which are **aggregations** of groups of individual values from a more extensive table within one or more discrete categories.

The aggregations or summaries of the groups of the individual terms might include sums, averages, counts, or other statistics.

Pandas' `pivot_table` function operates similar to a spreadsheet, making it easier to group, summarize and analyze your data. Here's how to create your own.

- A pandas pivot table has three main elements:
- **Index:** This specifies the row-level grouping.
- **Column:** This specifies the column level grouping.
- **Values:** These are the numerical values you are looking to summarize.

	normalized-losses	make	fuel-type	aspiration	num-of-doors
0	NaN	alfa-romero	gas	std	two
1	NaN	alfa-romero	gas	std	two
2	NaN	alfa-romero	gas	std	two
3	164.0	audi	gas	std	four
4	164.0	audi	gas	std	four

highway-mpg	price	target
27	13495.0	3
27	16500.0	3
26	16500.0	1
30	13950.0	2
22	17450.0	2

```
pivot = np.round(pd.pivot_table(data, values='price',  
                                index='num-of-doors',  
                                columns='fuel-type',  
                                aggfunc=np.mean),2)
```

A diagram illustrating a pivot table structure. The table has 'num-of-doors' as the index and 'fuel-type' as the columns. The values are rounded to two decimal places. Annotations include: 'Columns' pointing to the 'fuel-type' header, 'Index' pointing to the 'num-of-doors' header, and 'Values' pointing to the numerical data cells.

fuel-type			
		diesel	gas
num-of-doors	four	16432.38	13092.81
	two	14350.00	12762.76

```
np.round(pd.pivot_table(data, values='price',
                        index=['body-style'],
                        columns='num-of-doors',
                        aggfunc=[np.mean, np.median],
                        fill_value=0),2)
```

		mean		median	
num-of-doors		four	two	four	two
body-style					
hardtop	0.00	22208.50	0	19687.5	
wagon	12371.96	0.00	11694	0.0	
sedan	14614.13	14283.00	12555	8678.0	
hatchback	8372.00	10230.79	8073	8970.0	
convertible	0.00	21890.50	0	17084.5	

```
np.round(pd.pivot_table(data, values='price',
                        index=['body-style'],
                        columns=['num-of-doors'],
                        aggfunc=[np.sum],
                        fill_value=0,
                        margins=True, margins_name='Total'),2)
```

			sum
num-of-doors	four	two	Total
body-style			
hardtop	0	177668	177668.0
wagon	309299	0	309299.0
sedan	1139902	199962	1339864.0
hatchback	83720	593386	677106.0
convertible	0	131343	131343.0
Total	1532921	1102359	2635280.0

```
>>> df
```

	A	B	C	D	E
0	foo	one	small	1	2
1	foo	one	large	2	4
2	foo	one	large	2	5
3	foo	two	small	3	5
4	foo	two	small	3	6
5	bar	one	large	4	6
6	bar	one	small	5	8
7	bar	two	small	6	9
8	bar	two	large	7	9

```
>>> table = pd.pivot_table(df, values=['D', 'E'], index=['A', 'C'],
...                          aggfunc={'D': "mean",
...                                    'E': ["min", "max", "mean"]})
>>> table
```

		D		E	
		mean	max	mean	min
A	C				
bar	large	5.500000	9	7.500000	6
	small	5.500000	9	8.500000	8
foo	large	2.000000	5	4.500000	4
	small	2.333333	6	4.333333	2